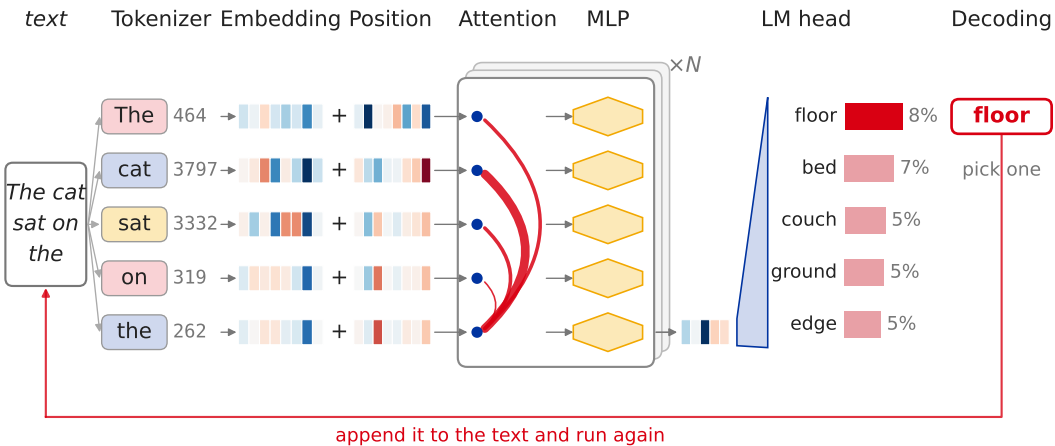
Pretraining

From the internet to a base model

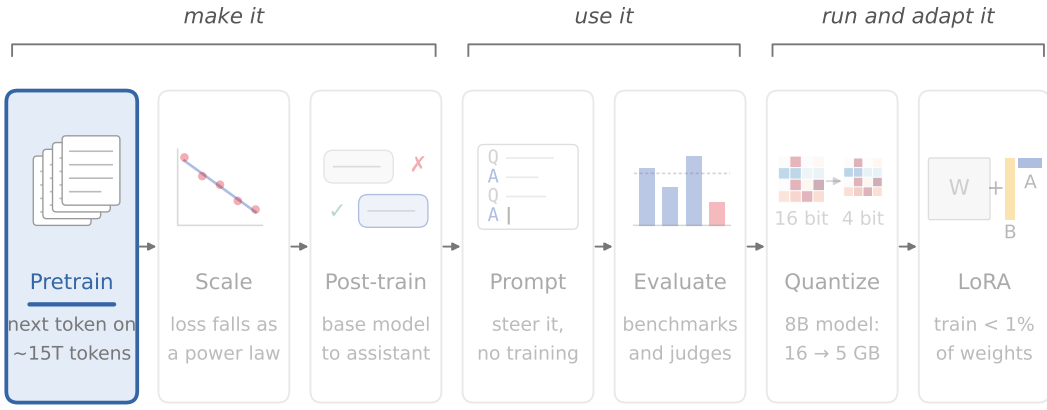
LLM chapter, session 7. Toy numbers unless a slide says otherwise.

Where we are: one forward pass



The forward pass is built (LLM-1 to LLM-6). Today: where its 124M numbers come from.

Where we are: the life of a model



Today: the first box. Everything after it starts from what it produces, a **base model**.

Finish these

Two famous openings. GPT-2 small gets each one and writes the next 48 tokens, always taking the most likely one (greedy, LLM-6).

A. "Permission is hereby granted, free of charge, to any person obtaining a copy . . . "

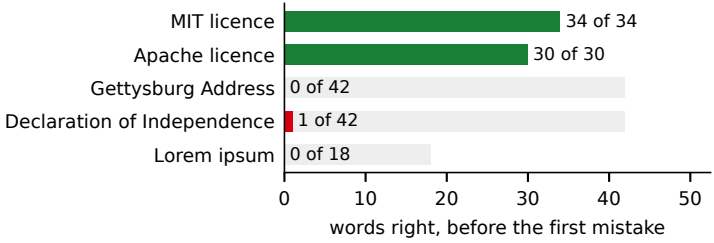
the MIT software licence

B. "Four score and seven years ago our fathers brought forth . . . "

the Gettysburg Address, 1863

Predict first: which one does it continue word for word? Both, one, or neither?

The licence, not the speech



A: all 34 words right, and the Apache licence 30 of 30. **B**: wrong from the first word ("the first of the great and great, . . . "); the Declaration of Independence fares no better. Lorem ipsum (0) is common on the web, but dataset filters often drop it (C4 removes every page that contains it).

Famous is not the same as frequent. A software licence is pasted into countless source files; a speech is quoted far less often. Today: where the 124M numbers come from, and why they memorised the licence and not the speech.

Cached GPT-2 small, greedy, 48 new tokens; words compared after collapsing whitespace.

Outline

The objective

One training step, by hand

The data

What comes out

The objective

Hide part of the text, and make the model guess it.

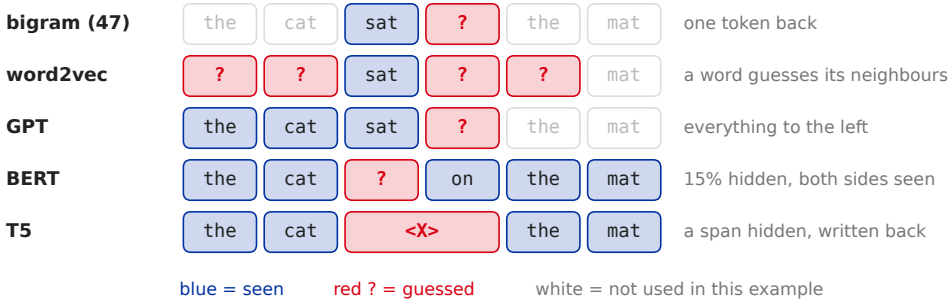
You have seen one already

- LLM-3: with the causal mask, one forward pass makes a guess at **every** position: the next token, from everything before it.
- Each guess is scored by $-\ln p(\text{the right token})$; the **loss** is the average over all the guesses.
- Gradient descent (ch11) nudges all 124M numbers so the right tokens become a little more likely. Repeat on the next batch, billions of times.

That is the whole objective. Most of today is about the text it runs on.

But is "guess the next token" the only way to turn raw text into a training signal?

Five ways to hide text



Each row is one training example on the same sentence. The model learns whatever it needs to fill the red cells. Next: word2vec, BERT and T5 (GPT's row is LLM-3's).

word2vec: a word and its neighbours



pairs: (sat, the) (sat, cat) (sat, on) (sat, the)

window 2: two words each side; every word takes a turn in the middle

- **Skip-gram** (Mikolov et al., 2013): every word guesses each neighbour in a window. "sat" gives 4 pairs; the whole six-word sentence gives 18.
- No word order inside the window, nothing beyond it, no network around it: just a table with one row of numbers per word - the kind of table LLM-2 showed.

Why word2vec learns meaning

Each word gets a vector. For a pair seen in the text, say (sat, cat):

- push the dot product $u_{\text{sat}} \cdot v_{\text{cat}}$ **up**;
- pick a few random words ("negative samples", say 5) and push their dot products with u_{sat} **down**.

$$\text{loss} = -\ln \sigma(u_{\text{sat}} \cdot v_{\text{cat}}) - \sum_{k=1}^{5} \ln \sigma(-u_{\text{sat}} \cdot v_k)$$

(σ is the sigmoid, and $-\ln \sigma(x)$ shrinks as x grows: minimising the first term raises $u_{\text{sat}} \cdot v_{\text{cat}}$, the second lowers the random ones. Each word keeps two vectors, u as the centre and v as a neighbour; the u's are the table you keep.)

"cat" and "dog" both sit next to "the", "fed" and "sat", so they are pushed towards the same places: meaning from the company a word keeps. Cheap and fast, but with no order and no depth it cannot write a sentence.

BERT: hide 15%, see both sides

- Masked language modeling (LLM-3), in detail: pick **15%** of the tokens (Devlin et al., 2018). Of those: 80% become `[MASK]`, 10% become a random token, 10% stay as they are. Predict the original at all of them.
- Why not always `[MASK]`? When the model is used, there is no `[MASK]` in the input; the 10% + 10% stop it from relying on one.
- Every other token is visible, **on both sides**: LLM-5's mask A.

So BERT cannot write left to right (LLM-5's cold open). What it is good at is the vectors: a sentence read in both directions. Embedding models for search descend from it (the RAG chapter).

And only the 15% are scored, so each sentence gives fewer training signals.

T5: hide whole spans

The T5 paper's own example (Raffel et al., 2020):

original	`Thank you for inviting me to your party last week.`
input	`Thank you <X> me to your party <Y> week.`
target	`<X> for inviting <Y> last <Z>`

- 15% of the tokens are dropped, in spans of 3 on average; each span becomes one **sentinel** token, and a final sentinel (<Z>) marks the end of the target.
- The encoder reads the input; the decoder writes the missing spans back (LLM-5's mask C).

Why next-token won

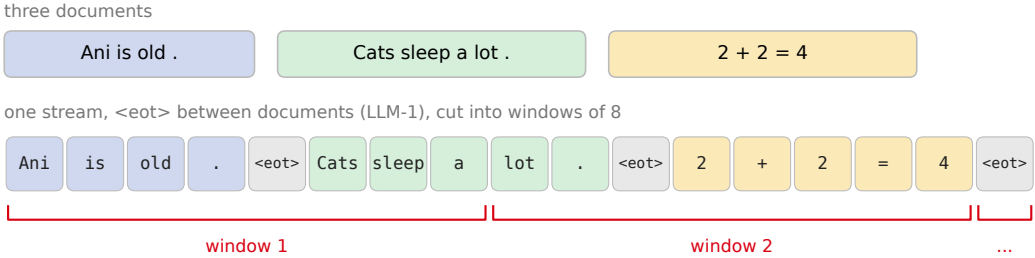
- **Every position is a target.** A 1,024-token window gives 1,023 guesses (LLM-4); on the same 1,024 tokens, BERT's 15% gives about 154.
- **It writes.** The model you pretrain is already the one that generates.
- **Any text will do.** No labels and no masking recipe: the next token is always sitting right there.

Not a cleverer objective: more signal per sentence, and a model that writes. LLM-5 said the same about the architecture.

One training step

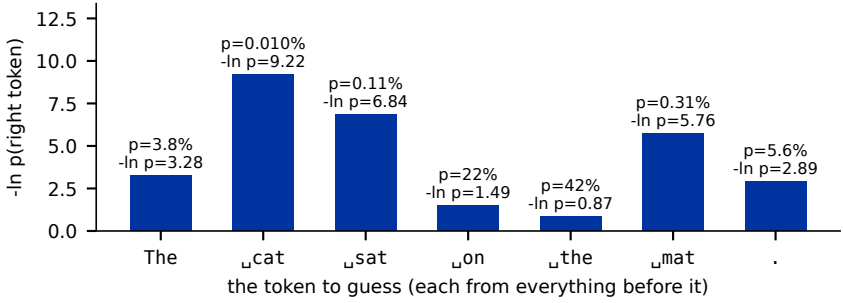
From three documents to one number.

Pack the documents



- Documents have every length; the GPU wants equal windows. So join them with the end-of-text token between (GPT-2's <|endoftext|>, LLM-1) and cut every 8 tokens (GPT-2: every 1,024). Toy tokens here: one word each.
- Both windows cross a document boundary. The <eot> tells the model that what follows is unrelated.

One window, seven guesses



Real GPT-2 on <eot>`The cat sat on the mat.`: 8 tokens, 7 guesses (the first token has nothing before it), all in one pass (LLM-4). (␣ = the leading space.)

By hand: loss $= (3.28 + 9.22 + 6.84 + 1.49 + 0.87 + 5.76 + 2.89)/7 = 30.35/7 =$ **4.34**

" cat" at 0.010% is the big surprise: 30% of the whole loss. Training pushes every right token up, and the biggest surprises carry the most loss.

Loss to perplexity

The name inventor (47) turned loss into something you can feel:

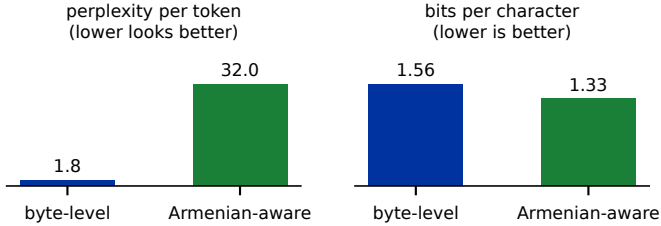
$$\text{perplexity} = e^{\text{loss}} = e^{4.34} \approx \mathbf{77}$$

- As unsure as if it picked among 77 equally likely tokens at every step. Guessing uniformly over GPT-2's vocabulary would be 50,257; a perfect model, 1.
- Seven guesses are a noisy estimate. Papers average over millions of tokens of held-out text.

Perplexity counts surprise *per token*. That is fine for one model on one text. It goes wrong as soon as the tokens are not the same size.

Perplexity across tokenizers lies

Toy numbers. One Armenian sentence of 45 characters (83 bytes: each letter is 2). Two tokenizers: byte-level pieces, 80 tokens and 70 bits of surprise in total; Armenian-aware, 12 tokens and 60 bits. (Bits are nats / $\ln 2$; $2^{\text{bits}} = e^{\text{nats}}$, the same perplexity.)



Per token: $70/80 = 0.875$ bits, perplexity $2^{0.875} = 1.8$; against $60/12 = 5$ bits, $2^5 = 32$. The byte-level model looks $17\times$ better. Per character: $70/45 = 1.56$ against $60/45 = 1.33$ bits. It is worse.

Short tokens are easy to guess one at a time (LLM-1's token tax). Compare **bits per character** (or per byte) on the same text, never per-token perplexity across tokenizers or languages.

The data

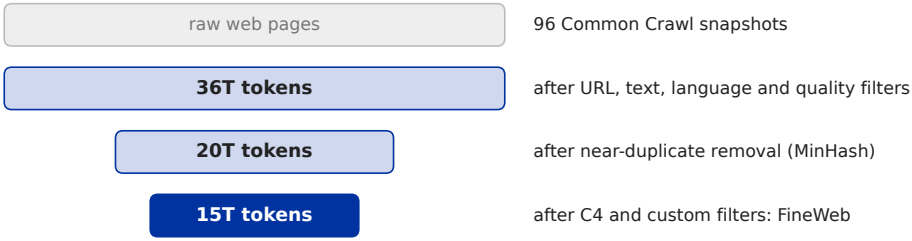
Most of the work is throwing text away.

How much text

GPT-2 (2019)	WebText: pages linked from Reddit posts with at least 3 karma (net upvotes); about 8 million documents, 40 GB. Wikipedia removed (it overlaps with test sets).
FineWeb (2024, open)	15T tokens of English, from 96 snapshots of Common Crawl, a non-profit that crawls the web and gives the pages away.
Llama 3 (2024)	trained on about 15.6T tokens; the Llama 3 paper puts Llama 2 at 1.8T.

In five years, from 40 GB of hand-picked links to trillions of tokens of filtered web. At that scale nobody reads the data. Rules and classifiers decide what a model sees.

The funnel



- The first filters, in order: block bad URLs; pull the main text out of the HTML; keep English (a classifier's confidence ≥ 0.65); drop low-quality and repetitive pages.
- Then remove near-duplicates (in three frames), apply most of C4's rules and FineWeb's own, and mask email and IP addresses.
- From 36T to 15T: more than half of what survived the first filters is thrown away.

Penedo et al. (2024), "The FineWeb Datasets", arXiv 2406.17557.

Clean this page by hand

Home | News | Contact

Please enable Javascript to view this page.

Yerevan is one of the oldest cities in the world.

It was founded in 782 BC as Erebuni.

Read more.

© 2026 All rights reserved

Four of C4's rules (the T5 paper, 2019):

1. a line must end in . ! ? or a closing quote;
2. a line needs at least 5 words;
3. a line that mentions Javascript goes;
4. a page needs at least 3 sentences.

Predict first: what is left of this page?

Two good sentences, and the page still goes

~~Home | News | Contact~~	drop: no end punctuation
~~Please enable Javascript to view this page.~~	drop: mentions Javascript
Yerevan is one of the oldest cities in the world.	keep
It was founded in 782 BC as Erebuni.	keep
~~Read more.~~	drop: fewer than 5 words
~~© 2026 All rights reserved~~	drop: no end punctuation

2 lines survive = 2 sentences; a page needs 3, so the whole page goes

Four lines fail a line rule. The two that pass are real content, but two sentences are fewer than three, so rule 4 removes the page.

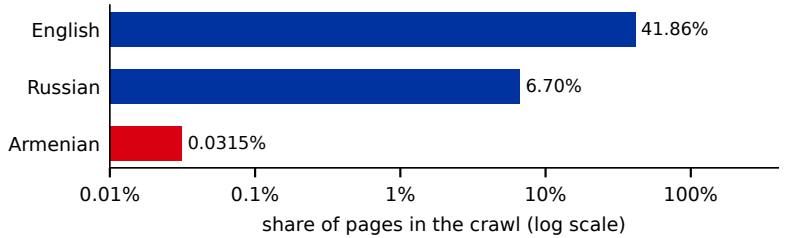
Rules like these are blunt. In FineWeb's tests the end-punctuation rule helped most on its own, but removed about 30% of all tokens, so FineWeb dropped it. Every rule trades the junk it removes against the good text it loses.

Duplicates

- The web repeats itself: mirrors, templates, boilerplate, licences. FineWeb compares documents by their sets of 5-word sequences (MinHash) and keeps one of each group that is at least 75% alike.
- Lee et al. (2022) found one 61-word sentence repeated over **60,000** times in C4. Trained on deduplicated data, models emitted memorised text **10**$\times$ less often.
- Carlini et al. (2022): memorisation grows log-linearly with the number of copies, with model size, and with the length of the prompt.

That is the cold open. The licence is the extreme duplicate; the speech is quoted in a few places. The model holds what it saw many times.

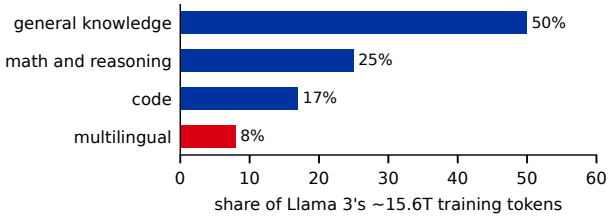
How much of the web is Armenian?



Common Crawl's latest crawl (CC-MAIN-2026-39), share of pages by main language: about **1,330 English pages for every Armenian one**. FineWeb-2, the multilingual sibling: 634 million Armenian words (1.76M documents) against 589 billion Russian words.

A model sees Armenian about as often as the web writes it. Few examples, plus LLM-1's token tax, are a large part of why answers in Armenian are weaker. And a language classifier decides what counts as Armenian in the first place.

The mixture



A lab does not train on the funnel's output as it is. It decides how much of each kind of text goes in, and can give scarce, valuable kinds (code, maths) more weight than their share of the web.

Llama 3's mix, over about 15.6T tokens.

Every language except English shares the red 8%. The mixture is a decision, and it shapes what the model is good at as much as the architecture does.

What comes out

A base model: it continues documents.

Watching it learn

Karpathy reproducing GPT-2 small (2025): 1 million tokens per step, 32,000 steps, about 33 billion tokens in all. Every 20 steps the model writes a sample.

- **Step 20:** completely random text. The network is still random.
- **About 1% in (step 420):** "since she is mine it's a part of the information should discuss my father great companions Gordon showed me sitting over at". Word pairs make sense; the sentence does not.
- **After the whole run** (the rest takes a day or two): fairly coherent English.

The loss falls smoothly; what the samples show is local structure first (which word follows which), then longer-range sense.

Karpathy, "Deep Dive into LLMs like ChatGPT" (2025), 35:00-38:00.

It recites, and it leaks

- **It recites what it saw many times**: the licences in the cold open.
- **You have seen this before.** The name inventor (47, section 12), trained 8× too long (1,200 epochs): the share of its "inventions" that were real training names climbed from 0% to 32%.
- **It can leak what it saw once.** Carlini et al. (2021) pulled hundreds of verbatim sequences out of GPT-2, including names, phone numbers and email addresses, each from a single training document.

Training data is not read and forgotten. That is why FineWeb masks email addresses, and why removing duplicates matters for privacy as well as quality.

It continues documents

a bare question

```
What is the capital of Armenia?

The capital of Armenia is the capital of
Armenia.

The capital of Armenia is the capital of
Armenia.

The capital of Armenia
```

a Q/A document with one worked example

```
Q: What is the capital of France?
A: Paris

Q: What is the capital of Armenia?
A: Armenia

Q: What is the capital of Azerbaijan?
```

black = the prompt; colour = what GPT-2 wrote (greedy)

Left: no answer, just more text about the question (here, a loop). Right: as a Q/A document the format works - a short answer, then the next question, because Q/A pages have many. The answer is wrong: 124M parameters do not hold "Yerevan".

A base model is a **document simulator**. To get an answer, the answer has to be the likely continuation. LLM-9 trains that behaviour in.

GPT-1, GPT-2, GPT-3

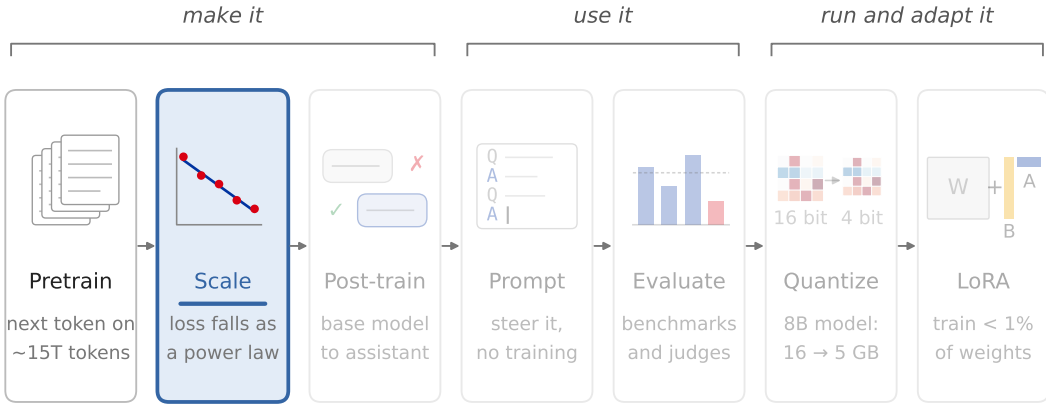
Model	Size	How it was used for a task
GPT-1 (2018)	117M	pretrain, then fine-tune a copy for each task
GPT-2 (2019)	1.5B	no fine-tuning: write the task into the text ("TL;DR:" after an article makes it summarise). The largest of four sizes; today's runs used the smallest, 124M
GPT-3 (2020)	175B	a few worked examples in the prompt: **few-shot**, no weight updates

The same objective all three times. What changed was scale, and how much of the task could move from training into the prompt. How the few-shot gain grows with size is LLM-8; how to write such prompts is LLM-10.

Recap

- **The objective**: hide text and guess it. Next-token won: every position is a target, and it writes. word2vec, BERT and T5 hide text in other ways.
- **One step**: pack documents with `<eot>`, cut windows, average $-\ln p$ of the right tokens. Perplexity $= e^{\text{loss}}$; across tokenizers compare bits per character, not per token.
- **The data**: most of the work is filtering (FineWeb: 36T $\to$ 20T $\to$ 15T tokens) and removing duplicates. Armenian is 0.03% of the pages; the mixture is a decision.
- **What comes out**: a base model, a document simulator. It recites what it saw often, can leak what it saw once, and continues instead of answering.

Next on the map



Next (LLM-8): how big a model, and how much data, for a given compute budget.

Sources & further reading

- Mikolov et al. (2013), "Distributed Representations of Words and Phrases and their Compositionality" (skip-gram, negative sampling).
- Devlin et al. (2018), "BERT"; Raffel et al. (2020), "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer" (T5, C4, span corruption).
- Radford et al. (2018, 2019), GPT-1 and GPT-2 (WebText); Brown et al. (2020), GPT-3.
- Penedo et al. (2024), "The FineWeb Datasets", arXiv 2406.17557; FineWeb-2 dataset card (Hugging Face); Common Crawl language statistics, CC-MAIN-2026-39.
- Lee et al. (2022), "Deduplicating Training Data Makes Language Models Better"; Carlini et al. (2021), "Extracting Training Data from Large Language Models"; Carlini et al. (2022), "Quantifying Memorization Across Neural Language Models".
- Llama Team (2024), "The Llama 3 Herd of Models"; Karpathy (2025), "Deep Dive into LLMs like ChatGPT".

Figures: `py_src/pretraining_figs.py`. Real: GPT-2's recitations, its loss on one window, its two answers (cached model, one run), and the published numbers above. Toy: the objectives, packing, word2vec pairs, the tokenizer comparison, the web page.